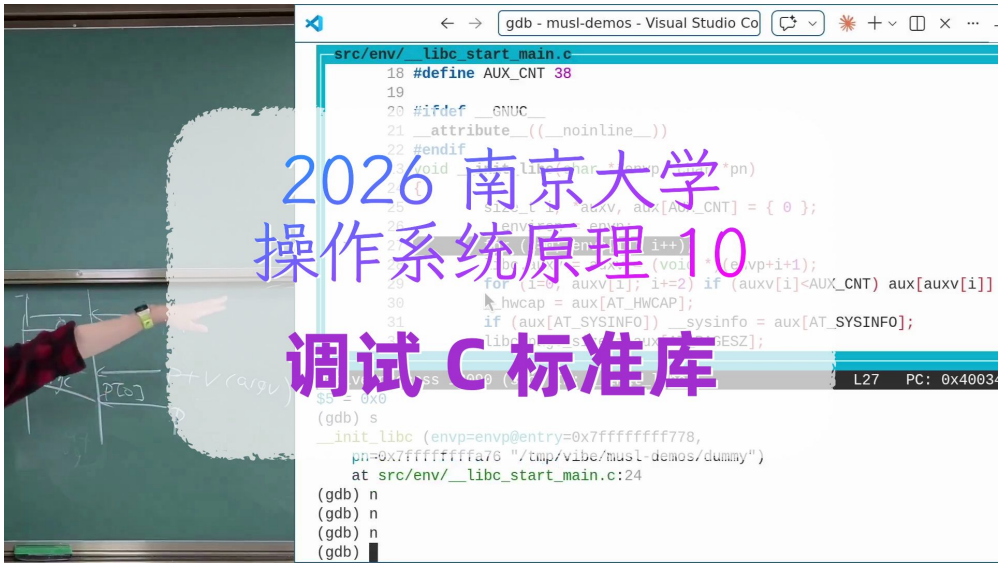
课程笔记

调试 C 标准库

南京大学 操作系统原理 2026 春季学期 第 10 讲

lecture-to-notes

2026 年 4 月 5 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）
发布日期：2026 年春季学期
视频时长：1 小时 21 分钟
视频链接：https://www.bilibili.com/video/BV1yu9cBCEAb

目录

1 调试信息 **2**
1.1 使用自己的 libc 2
1.2 DWARF 调试信息格式 2
1.3 本章小结 3

2 调试信息的作用 **3**
2.1 Debug Info 能做什么 3
2.2 AI 辅助调试 4
2.3 本章小结 4

3 调试 crt1, printf, setjmp, vdso **5**
3.1 GDB 调试 printf 5
3.2 调试 setjmp / longjmp 6
3.3 AI 分析 instrumentation 日志 7
3.4 本章小结 7

4 总结与延伸 **7**
4.1 讲者的核心信息 7
4.2 结构化提炼 8
4.3 与第 09 讲的关联 8
4.4 拓展阅读 8

1 调试信息

1.1 使用自己的 libc

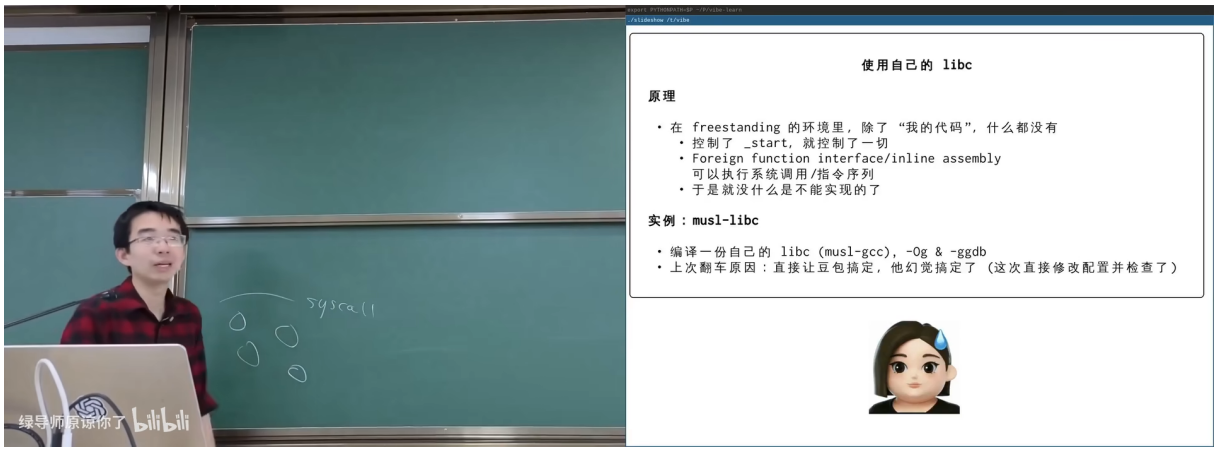


图 1: 使用自己的 libc：freestanding 环境 + musl-libc 编译[1]

为什么要 “使用自己的 libc”？

系统自带的 libc（glibc）是**没有调试信息**的——编译时用了 `-O2` 优化且 strip 了符号。想用 GDB 单步调试 `printf` 的实现？做不到。

解决：自己编译一份 musl-libc（`musl-gcc -O0 -ggdb`），保留完整的调试信息（DWARF symbols、源码行号映射、变量名）。这样就能在 GDB 里逐行查看 `printf`、`malloc`、`_start` 的实现。

1.2 DWARF 调试信息格式

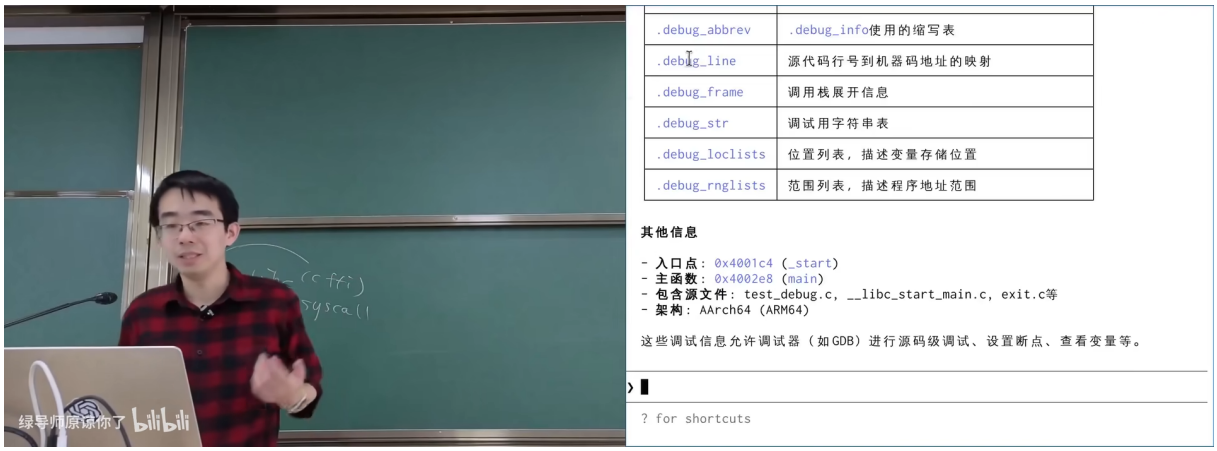


图 2: DWARF 调试信息的各个 section：.debug_abbrev, .debug_line, .debug_frame 等[2]

[1] 视频画面时间区间：00:02:00–00:02:30。帧实际内容：slide “使用自己的 libc”——在 freestanding 环境中，除了 “裸的代码” 什么都没有，甚至没有 _start。通过 FFI/inline assembly 可以直接和机器交互。实例：musl-libc，编译一份自己的 libc (musl-gcc, -O0 -ggdb)。

DWARF 调试信息的核心 sections

- `.debug_line`：源代码行号 ↔ 机器码地址的映射——GDB “next” 的基础
- `.debug_frame`：栈帧展开信息——backtrace 的基础
- `.debug_info`：变量名、类型、作用域——GDB “print x” 的基础
- `.debug_loclists`：变量在寄存器/内存中的位置——优化代码中变量 “跑到了哪里”

这些调试信息**不影响程序运行**——它们只是 ELF 文件中额外的 section，被调试器（GDB）读取。

1.3 本章小结

调试信息（DWARF）是连接源代码和机器码的桥梁。自己编译 musl-libc 保留调试信息，就能用 GDB 单步调试标准库的每一行实现。

2 调试信息的作用

2.1 Debug Info 能做什么

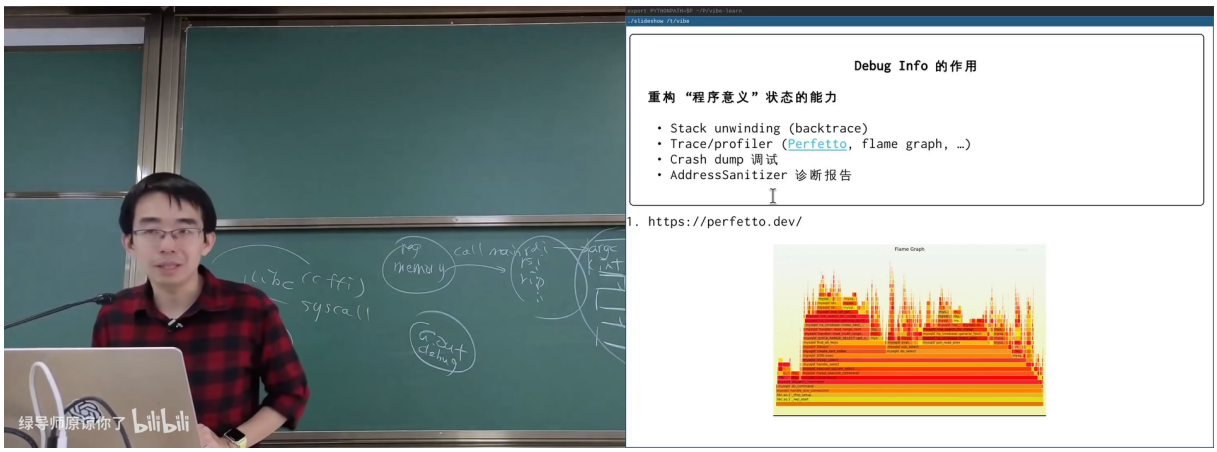


图 3: Debug Info 的作用：重构 “程序语义” 状态的能力[3]

[2]视频画面时间区间：00:12:00–00:12:30。帧实际内容：slide 展示 DWARF 调试信息的 section 列表——.debug_abbrev（使用的缩写表）、.debug_line（源代码行号到机器码地址的映射）、.debug_frame（调用栈展开信息）、.debug_str（调试用字符串表）、.debug_loclists（位置列表，描述变量存储位置）、.debug_rnglists（范围列表，描述程序地址范围）。以及其他 ELF 信息：入口点 0x400160 (_start)、主函数 0x4002e0 (main)。

[3]视频画面时间区间：00:31:29–00:31:59。帧实际内容：slide “Debug Info 的作用”——重构 “程序语义” 状态的能力。包括 Stack unwinding (backtrace)、Tracer/profiler (Perfetto, flame graph, ...)、Crash dump 调试、AddressSanitizer 金属报告。附 Perfetto flame graph 截图。

调试信息的四大用途

1. **Stack unwinding (backtrace)**：程序崩溃时显示完整的调用链——从 crash 点回溯到 main
2. **Profiler / Flame Graph**：性能分析工具需要知道“当前在执行哪个函数”——通过 `.debug_frame` 展开栈帧
3. **Crash dump 调试**：core dump 文件 + 调试信息 = 事后分析崩溃原因
4. **Sanitizer 报告**：AddressSanitizer / ThreadSanitizer 的错误报告需要源码行号

2.2 AI 辅助调试

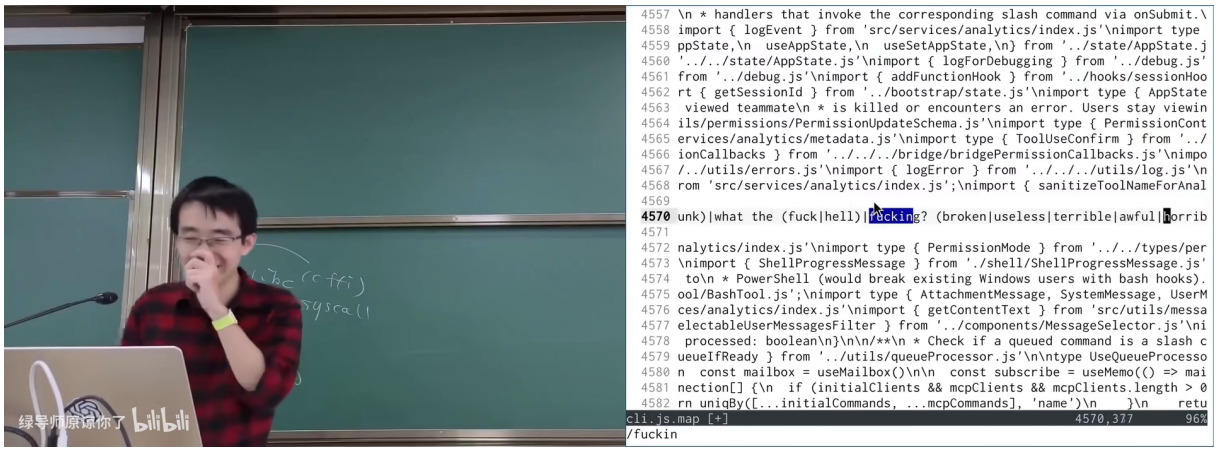


图 4: Claude Code 分析日志时遇到的真实错误——“what the f**k!!”[4]

AI 时代的调试

jyy 展示了 Claude Code 分析日志的实际场景——AI 也需要“调试信息”来理解代码行为。不管是人还是 AI，调试的本质都是：**从执行痕迹（日志、core dump、trace）重构程序的语义状态**。调试信息（DWARF）就是让这种重构成为可能的“翻译表”。

2.3 本章小结

调试信息让我们能从机器状态重构“程序语义”。核心能力：backtrace（栈展开）、profiling（火焰图）、crash dump 分析、sanitizer 报告。AI 也依赖同样的机制来理解程序行为。

[4] 视频画面时间区间：00:24:00–00:24:30。帧实际内容：VS Code 终端显示大量 import 语句和一行突出的错误“4570 unk) what the f**k!!

(brokenUsesTerriblyBadFull0rrib...”——Claude Code 在分析一个复杂项目时遇到的真实报错。jyy 用这个例子说明 AI 也需要调试信息来理解错误。

3 调试 crt1, printf, setjmp, vdso

3.1 GDB 调试 printf

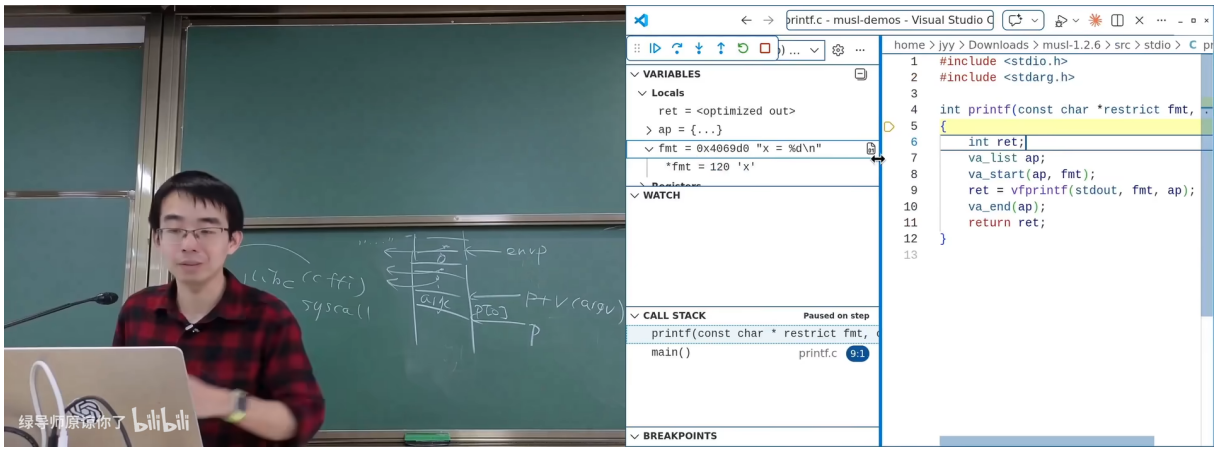


图 5: GDB 调试 printf：VS Code 中单步进入 musl libc 的 printf 实现[5]

```
int printf(const char *restrict fmt, ...) {
    int ret;
    va_list ap;
    va_start(ap, fmt);
    ret = vfprintf(stdout, fmt, ap);
    va_end(ap);
    return ret;
}
```

Listing 1: musl libc 中 printf 的真实实现

用 GDB 亲眼看到 printf 的实现

通过自编译的 musl-libc + GDB，你可以：

1. 在 `printf` 处设断点，单步进入
2. 看到 `va_list` 如何展开可变参数
3. 跟踪 `vfprintf` → `__stdio_write` → `write` syscall 的完整调用链
4. 在 VARIABLES 面板中观察每个中间变量的值

这比任何文档都直观——“Read the Source, Luke” 的终极版本。

[5] 视频画面时间区间：00:42:03–00:42:33。帧实际内容：VS Code debugger 界面，左侧 VARIABLES 面板显示 ret, optimized out，CALL STACK 显示 printf(const char *restrict fmt, ...) → main()。右侧源码显示 musl 的 printf.c 实现：int printf(const char *restrict fmt, ...) { int ret; va_list ap; va_start(ap, fmt); ret = vfprintf(stdout, fmt, ap); ...}。

3.2 调试 setjmp / longjmp

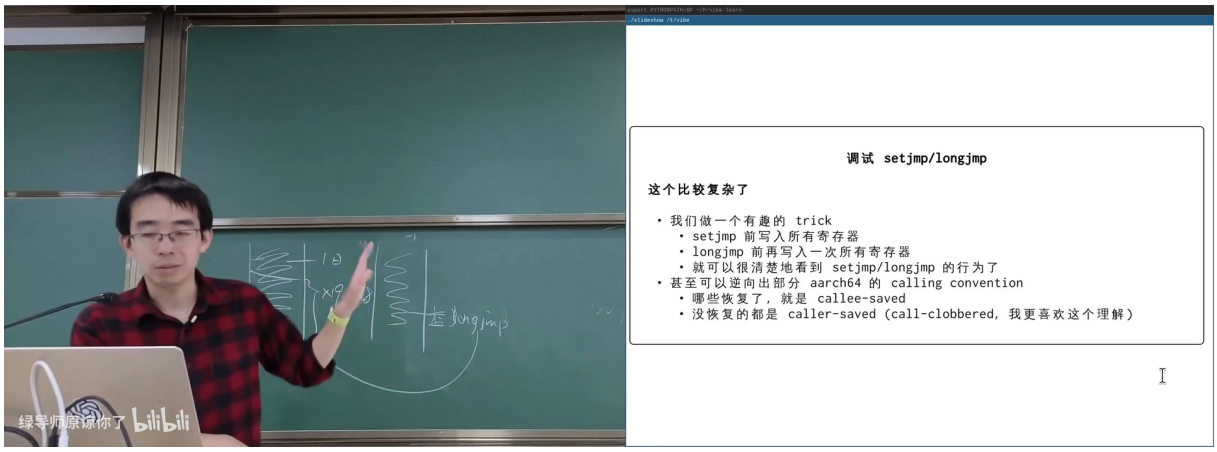


图 6: 调试 setjmp/longjmp：setjmp 写入所有寄存器，longjmp 恢复[6]

用寄存器染色理解 calling convention

jyy 的巧妙实验：在 `setjmp` 前把所有寄存器设为已知值（“染色”），`longjmp` 后检查哪些寄存器被恢复了：

- **恢复了的** = callee-saved 寄存器（`rbx`，`rbp`，`r12-r15`）
- **没恢复的** = caller-saved / call-clobbered 寄存器（`rax`，`rcx`，`rdx`，`...`）

这比背 ABI 表格直观 100 倍——`setjmp/longjmp` 的行为**就是** calling convention 的物理体现。

[6]视频画面时间区间：00:54:33–00:55:03。帧实际内容：slide “调试 setjmp/longjmp”——这个比较复杂了。我们做一个有趣的 trick：setjmp 前写入所有寄存器、longjmp 恢复入另一次所有寄存器。这可以很直观地看到 setjmp/longjmp 的行为了。其实可以很好地映射到 saved/calling convention：哪些恢复了就是 callee-saved、没恢复的就是 caller-saved (call-clobbered, 我更喜欢用这个理解)。

3.3 AI 分析 instrumentation 日志

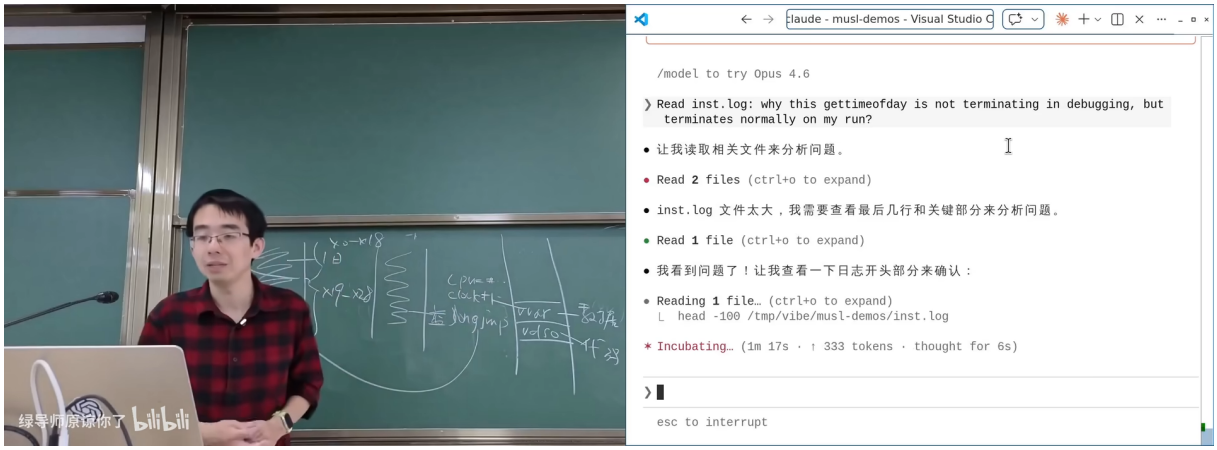


图 7: 让 Claude Opus 4.0 分析 inst.log："让我读取相关文件来分析问题"[7]

AI + Instrumentation

jyy 现场演示了让 Claude Opus 4.0 分析程序的 instrumentation 日志（`inst.log`）。AI 的优势：

- 能处理**巨大的日志文件**——人类看不完的 trace，AI 可以快速扫描
- 能**关联多个信息源**——同时看代码、日志、配置
- 能**提出假设并验证**——"让我查看一下日志开头部分来确认"

但 AI 仍然需要调试信息来理解日志的含义——没有 DWARF，`inst.log` 就是一堆无意义的地址。

3.4 本章小结

亲手调试 C 标准库是理解它的最佳方式。printf 的调用链（printf → vfprintf → ___stdio_write → write syscall）、setjmp/longjmp 的寄存器保存/恢复（= calling convention 的物理体现）、AI 辅助分析 instrumentation 日志——这些都是"看代码"无法替代的体验。

4 总结与延伸

4.1 讲者的核心信息

$$\text{调试 C 标准库} = \text{自编译 musl-libc} + \text{DWARF 调试信息} + \text{GDB 单步} + \text{AI 辅助分析}$$

[7] 视频画面时间区间：01:09:33–01:10:03。帧实际内容：Claude Code 界面，Claude 正在分析 inst.log 文件——"让我读取相关文件来分析问题"，Read inst.log（"inst.log 文件太大，我需要查看它的关键部分来分析问题"），Reading 2 files，"我看到问题了！让我查看一下日志开头部分来确认"，Incubating (im 51s → 133 tokens thought for 0s)。

4.2 结构化提炼

四个核心 Takeaway

1. **调试信息是连接源码和机器码的桥梁**——DWARF 的 `.debug_line` / `.debug_frame` / `.debug_info` 让 GDB 能显示源码行号、展开调用栈、打印变量值
2. **自编译 musl-libc 是调试标准库的前提**——系统 glibc 被 strip 了调试信息，自己编译 musl 加 `-O0 -ggdb` 才能单步调试
3. **setjmp/longjmp 的寄存器染色实验**——比任何文档都直观地展示了 calling convention：恢复的 = callee-saved，没恢复的 = call-clobbered
4. **AI + 调试信息 = 新的调试范式**——Claude Code 能分析巨大的 instrumentation 日志，但仍然依赖 DWARF 将地址翻译为语义

4.3 与第 09 讲的关联

第 09 讲讲了 C 标准库的三层封装（指令集/syscall/环境）。第 10 讲教你如何**亲手验证**这些封装——用 GDB 进入 printf 看到 vfprintf → write syscall 的完整链路，用寄存器染色验证 setjmp 保存了什么。

第 09 讲：C 标准库是什么 $\xrightarrow{\text{第 10 讲}}$ **亲眼看到**它如何工作

4.4 拓展阅读

- musl libc 源码: `https://musl.libc.org/`
- DWARF 标准: `https://dwarfstd.org/`
- Perfetto 性能分析工具: `https://perfetto.dev/`
- jyy Wiki: `https://jyywiki.cn/`